Step 5 — Il training (LoRA sul decision model; traccia encoder per i controlli)

Obiettivo

Addestrare un adapter LoRA sul decision model (JPT-9B per iterare, Winnow-12B per la versione finale) con una sola domanda, la loss di 04 e il curriculum a due stadi di 02e. In appendice (§9) la configurazione dell'encoder stile FactCG, usato come controllo E1 e come studente della distillazione.

Input e output

Input	pesi (Winnow: `winnow-12b-bf16/model/`; JPT: `kirp/jpt-9b` revisione `b447cc7`), dataset renderizzati per stadio, configurazione
Output	`finetune/outputs/<esperimento>/{stage1,stage2}/`: adapter per checkpoint, `config.json`, log, curve
Codice	`finetune/train.py` (**da scrivere**, condiviso con il piano generale; qui senza il ramo replay)

1. Configurazione di partenza

Parametro	Stadio 1	Stadio 2	Perché
Moduli LoRA	q, k, v, o, gate, up, down	idem (stesso adapter, si continua)	il compito tocca sia dove guardare sia cosa concludere
Rank / alpha	16 / 32	idem	sufficiente per un compito stretto; F3 prova 8/32/64
Dropout LoRA	0,05	0,05	
Learning rate	**5e-5**, warmup 3%, cosine	**1e-5**, warmup 3%, cosine	come MiniCheck (5e-5 poi 1e-5). Più alto del piano generale (2e-5) perché non c'è oblio da evitare; P1 prova 2e-5
Epoche	1	1	dati in parte sintetici: la seconda epoca sovra-adatta lo stile
Batch effettivo	32	32	
Lunghezza massima	4.096 (pilota) / 8.192	8.192	lo stadio 2 ha i documenti lunghi
Ottimizzatore	AdamW, weight decay 0	**nuovo** AdamW (come FactCG)	lo stato dell'ottimizzatore dello stadio 1 porterebbe il momento verso i dati sintetici
Precisione	bf16, gradient checkpointing	idem	
Loss	CE sulle lettere	idem	04
Checkpoint	ogni 20%	ogni 10%	
Seed	fisso; 3 seed per la configurazione finale		

Il pilota (`fc_pilot`) è a uno stadio con lr 2e-5: serve solo a verificare che tutto funzioni.

2. Hardware

Macchina	JPT-9B	Winnow-12B
Portatile RTX 5090, 24 GB	QLoRA a 4 bit, 4k token, batch fisico 1	no
L40S 45 GB	LoRA bf16, fino a 8k token	LoRA bf16, 4–8k token (pesi ~22 GB + attivazioni)
2× L40S	data parallel	data parallel

QLoRA va bene per iterare; i numeri definitivi si misurano sui pesi bf16 fusi.

3. Tempi (stima da verificare)

Una domanda per riga e niente replay: un terzo degli esempi rispetto alla modalità S3 del piano generale.

Corsa	Esempi	Token (≈)	Tempo su 1 L40S a ~1.000 token/s
`fc_pilot`, JPT	~25k	~15M	~4 h
`fc_stage1` + `fc_stage2`, JPT	~90k	~70M (stadio 2 più lungo)	~18–20 h
idem, Winnow	~90k	~70M	~25–30 h, o metà su 2× L40S

Misurare la velocità reale nei primi 100 passi.

4. Il ciclo

```
per stadio in [1, 2]:
    carica l'adapter dello stadio precedente (se c'è), ottimizzatore e scheduler nuovi
    per ogni micro-batch:
        forward fino allo stato nascosto finale
        h = stato in answer_pos;  s = h · (w_vero − w_falso) / T     (solo 2 righe della testa)
        loss = CE(s, y) [+ β·KL] [+ μ·L_pair]
        backward; ogni 32 esempi passo dell'ottimizzatore
    ogni checkpoint: salva l'adapter e valuta (§5)
```

5. Cosa controllare durante il training

Metrica	Su cosa	Atteso	Allarme
Loss di training	batch correnti	scende e si appiattisce	oscillazioni → lr troppo alto
BAcc, AUC	`dev_select`	sopra la baseline a una domanda	una fonte crolla mentre le altre salgono
Soglia ottimale	`dev_select`	scende da 0,90–0,95 verso 0,4–0,6	resta > 0,85 dopo il 50% dello stadio 1 → negativi "non stabilito" insufficienti
BAcc a soglia 0,5	`dev_select`	vicina a quella a soglia fittata	distanza > 1 punto → calibrazione da sistemare
ECE	`dev_select`	stabile o in calo	sale molto → sovra-confidenza (Brier, smoothing)
Solo claim	200 righe di `dev_select` con documento vuoto	P media bassa	sale → scorciatoie sul claim

Metrica	Su cosa	Atteso	Allarme
Righe lunghe	righe di `dev_select` > 2k token	BAcc in crescita nello stadio 2	calo nello stadio 2 → problema di chunking o troncamento

6. Test prima del primo training

Test	Cosa verifica
Margine ridotto = margine completo	`h · (w_vero − w_falso)` coincide con `z_vero − z_falso` del forward completo (eventuale softcapping/normalizzazione inclusi), differenza < 1e-3
Passo zero = baseline	prima di qualsiasi aggiornamento, P(supportato) su 200 righe = quelle delle cache della baseline (stesso prompt)
Overfit di 64 esempi	la loss va quasi a 0 in poche centinaia di passi: la pipeline impara

7. Scelta del checkpoint

1. Fra i checkpoint dello stadio 2, quello con BAcc macro più alta su `dev_select`, a parità (entro 0,3) quello con ECE più bassa.
2. Conferma su `dev_confirm`.
3. LLM-AggreFact non entra nella scelta.

8. Problemi tipici

Sintomo	Causa probabile	Cosa fare
La loss non scende	lettere o target invertiti (A = falso in Winnow, A = sì in JPT)	test di 03 §6
BAcc giù dopo pochi passi	lr troppo alto per Winnow; formato diverso dal benchmark	lr 2e-5; confronto token per token
Soglia ottimale bloccata in alto	pochi negativi "non stabilito"	più `remove_evidence` e C2D negativi
Ottimo in stadio 1, peggio dopo stadio 2 su test_id sintetici	atteso: lo stadio 2 sposta verso testi reali	giudicare su `dev_select`, non sul sintetico
Probabilità tutte 0/1	sovra-confidenza	Brier, smoothing, β·KL, temperatura rifittata

9. Appendice: traccia encoder stile FactCG (controllo E1, studente S)

Configurazione presa da FactCG-DBT (../models/FactCG.md, §4.3 e §5):

Parametro	Valore
Backbone	`microsoft/deberta-v3-large` (alternativa da provare: `answerdotai/ModernBERT-large`, contesto 8k)
Testa	`ContextPooler` + `Linear(1024→2)`, nuovi
Input	template FLAN a sequenza singola (03 §5), massimo 2.048 token

Parametro	Valore
Training	full fine-tuning, 2 stadi come sopra, lr 1e-5, warmup 6%, AdamW ε = 1e-6, batch effettivo 32–64
Precisione	bf16 (fp16 nel paper)
Inferenza	chunk 550 token, `max`, soglia 0,5 e fittata
Hardware	RTX 5090 24 GB, batch fisico 2 a 2.048 token con gradient checkpointing
Tempo	1–4 h per stadio sui nostri mix (FactCG: 18 min su 4 GPU per i suoi dati, più piccoli)

Per lo studente della distillazione il target è `p_insegnante` (probabilità morbida), con loss `KL(p_insegnante ‖ p_studente)` più, opzionalmente, CE sull'etichetta vera con peso 0,5 (08 §5).

10. Riproducibilità

`config.json` per esperimento: hash dei mix e dei dataset renderizzati, revisione dei pesi, domanda (`question.json`), iperparametri per stadio, seed, versioni delle librerie, macchina, durata.

Criteri per passare allo step successivo

- Un checkpoint che batte la baseline a una domanda su `dev_select` (confermato su `dev_confirm`), con soglia ottimale fra 0,3 e 0,7 e ECE non peggiore.